

Calibrated Ensemble Learning for Outcome Prediction

Sean Rockwitz
Independent Research
sean@rockwitz.com

Abstract—This paper presents the modeling strategy, the underlying mathematics, and the measured results behind a live, gradient-boosted ensemble system for binary outcome prediction. The strategy combines an Elo-based strength prior, exponential recency weighting, a three-learner stacked ensemble (XGBoost, LightGBM, CatBoost) with a learned logistic meta-learner, isotonic probability calibration with a variance-preserving blend step, and a domain-adapted feature pipeline that rescales its sample-size assumptions to fit datasets of very different sizes. We give the full mathematical treatment behind each design choice: the regularized split-gain criterion that governs how boosted trees grow, the effective-sample-size argument behind our recency weighting, the optimizer’s-curse argument behind how we size hyperparameter search spaces, the stacking objective that lets the ensemble automatically weight its base learners, and the isotonic calibration and blending equations that keep predicted probabilities both accurate and informative. We then report results: across our three most mature models, log-loss ranges from 0.6418 to 0.6624 and Brier score from 0.2179 to 0.2348, and our newest, smallest-data model currently posts the lowest log-loss of the three (0.6418) alongside 86.7% accuracy on its highest-confidence predictions ($n = 1,520$). We conclude that a domain-adapted feature strategy, grounded in the same shrinkage and effective-sample-size mathematics used throughout the rest of the pipeline, is what closes the gap between a smaller dataset and a larger one.

Index Terms—gradient boosting, stacked generalization, probability calibration, hyperparameter optimization, ensemble learning, model evaluation

I. INTRODUCTION

This paper is organized around three questions, in order: what is the modeling strategy, what is the mathematics behind it, and what does it actually achieve. We built a live outcome-prediction system around a shared modeling core: an Elo-style rating engine, exponential recency weighting, a three-model gradient boosting stack combined by a learned meta-learner, and isotonic probability calibration. Every design choice in that core has a specific mathematical justification, and we lay each one out in full rather than treating the architecture as a black box that happens to work.

The system predicts the probability that a given team wins a given game, from publicly observable statistics. We mention the application once, plainly, because the interesting content here is the modeling strategy and the mathematics behind it, not the domain it happens to be applied to.

This paper’s contributions:

- 1) A complete description of the modeling strategy: an Elo-based strength prior, recency-weighted training, a

stacked gradient boosting ensemble with a learned meta-learner, isotonic calibration with a variance-preserving blend, and a domain-adapted feature pipeline for smaller datasets.

- 2) The full mathematics behind each strategic choice: the regularized split-gain criterion, the effective-sample-size argument for recency weighting and for search-space sizing, the stacking objective, and the calibration and blending equations.
- 3) Measured results across our model portfolio, including a case study showing that the domain-adapted feature strategy currently produces the best-calibrated model we run, despite the smallest training set.
- 4) A summary of which findings we believe generalize beyond this particular system.

II. BACKGROUND

A. Gradient Boosting and Its Regularization Surface

Gradient boosted trees [1], [2], [3] build an additive model by repeatedly fitting a new tree to the residual of the current ensemble. Every split in every tree is only taken if it clears a minimum-gain threshold, the mechanism behind the regularization strategy in Section III-C.

B. Hyperparameter Search as Estimation Under Noise

Automated hyperparameter search treats each trial’s cross-validation score as an estimate of that configuration’s true generalization performance. When many configurations are compared under noise, the configuration with the best observed score is, in expectation, better estimated than it truly is, a selection-bias effect known in decision analysis as the optimizer’s curse [4], related to the "garden of forking paths" critique of data-dependent analysis choices [5]. This motivates how we size our search spaces (Section III-D).

C. The Curse of Dimensionality at Fixed Sample Size

Adding features without adding rows does not monotonically help a classifier. For a fixed training sample size, expected classifier accuracy as a function of feature count first rises and then falls, a peaking effect known as the Hughes phenomenon [6]. This motivates a pruning step in our feature strategy (Section III-D).

D. Effective Sample Size Under Weighting

When training examples are weighted rather than treated equally, the effective number of independent observations is smaller than the raw row count. Kish’s effective sample size [7] formalizes this, and it is the mathematical backbone of both our recency weighting strategy (Section III-B) and our domain-adapted feature strategy for smaller datasets (Section III-G).

E. Stacked Generalization

Combining several base models by fitting a meta-learner on their held-out predictions [8] generally outperforms any fixed-weight average of the same base models, because the combination weights are themselves learned from data the base models never saw. This is the strategy behind Section III-E.

F. Probability Calibration

A model can rank outcomes well and still be badly calibrated: its stated probabilities need not match observed frequencies for it to still separate positives from negatives reasonably well [10]. Isotonic regression, fit with the Pool-Adjacent-Violators algorithm [9], is the standard non-parametric fix, and Section III-F covers the calibration strategy built on top of it. We track Expected Calibration Error [11] and Brier score [12] throughout.

G. Shrinkage Estimation

When a per-entity statistic is estimated from few observations, a shrinkage estimator that pulls the noisy estimate toward a population mean [13] generally outperforms the raw maximum-likelihood estimate. This partial-pooling logic [14] underlies the domain-adapted feature strategy in Section III-G.

III. MODELING STRATEGY

This section lays out the strategic design of the system: what each component is for and why it takes the form it does. Section IV then gives the complete mathematics behind every choice made here.

A. An Elo Prior Instead of a Static Rating

Rather than treat team strength as a static, periodically retrained value, we track it with a continuously updated Elo rating that replays every completed game up to the prediction cutoff. This gives the ensemble a smoothly updated, leakage-free strength estimate as a single input feature, rather than asking the gradient boosting model to learn strength dynamics from raw box scores on its own.

B. Recency Weighting Tuned to Each Application’s Volatility

Because team strength drifts over a season, training examples are weighted by recency rather than treated as equally informative regardless of age. Critically, the weighting strength is tuned per application rather than fixed globally, because different applications have different within-season volatility, and the right amount of recency weighting is an empirical balance rather than a single global constant (Section IV-B).

C. Constrained Regularization Search

Regularization hyperparameters are searched within bounds chosen specifically to keep the model’s splitting behavior well away from a mathematical threshold beyond which trees stop growing altogether (Section IV-C). Staying inside that boundary by design, rather than trusting an unconstrained search to avoid it by luck, is a deliberate strategic choice.

D. Search Spaces Sized for Generalization, Not for the Best Reported Score

Hyperparameter search spaces are kept deliberately narrow rather than as wide as computationally affordable. A wider search reports a better cross-validation number more often, but Section IV-D formalizes why that number becomes less trustworthy as the search widens. Feature sets are grown the same way: new features are added in small batches, each followed by an importance-based pruning pass, rather than added in bulk on the assumption that anything correlated with the target in isolation is worth keeping.

E. A Learned Combination of Base Learners

Three gradient boosting libraries (XGBoost, LightGBM, CatBoost) are each tuned independently and combined with a logistic regression meta-learner rather than a fixed-weight average. This lets the combination step itself decide, from held-out data, how much weight each base learner deserves, including assigning close to zero weight to a base learner that is not contributing information beyond what the others already capture (Section IV-E).

F. Calibration That Preserves Predictive Variance

Rather than use isotonic calibration’s output directly, we blend it with the ensemble’s raw, continuous score. The reasoning (Section IV-F) is that isotonic regression’s output is piecewise constant by construction, and blending keeps the calibrated probability both well-calibrated in aggregate and informative at the level of an individual prediction.

G. A Feature Pipeline Rescaled to Each Dataset’s True Size

The same modeling core runs across applications of very different data volume. Rather than reuse one application’s feature parameters unchanged on a much smaller one, every sample-size-dependent parameter (rolling windows, minimum-sample thresholds, lookback windows) is rescaled to the smaller application’s real sample size, and its own true population averages are used as fallback defaults wherever a per-entity estimate is under-sampled (Section IV-H).

IV. MATHEMATICS

A. Elo Updates

Given ratings R_A, R_B before a game, the win probability for A is:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}. \quad (1)$$

After the outcome $S_A \in \{0, 1\}$ is known, both ratings update with a fixed step size K :

$$R'_A = R_A + K(S_A - E_A). \quad (2)$$

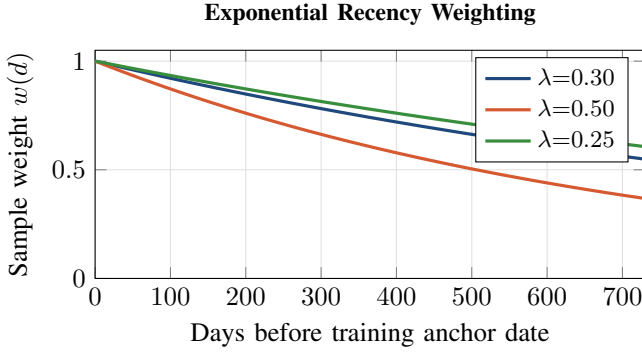


Fig. 1. Exponential recency weighting (Eq. 3) at three decay rates. Larger λ discounts older data faster, at the cost of a smaller effective sample size (Eq. 4).

B. Recency Weighting and Effective Sample Size

Every training row is weighted by an exponential decay anchored to the most recent date in the training set:

$$w(d) = \exp\left(-\lambda \frac{d}{365}\right), \quad (3)$$

where d is days before the anchor date. The effective sample size under this weighting [7] is:

$$n_{\text{eff}} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}. \quad (4)$$

Increasing λ reduces staleness bias but also shrinks n_{eff} , which increases the variance of every rolling feature estimated from the weighted sample. The right value of λ sits at the point where these two effects balance, and that point depends on how volatile within-season form actually is for a given application: a higher-volatility application tolerates a larger λ before variance costs start to outweigh the bias reduction, while a lower-volatility one does not.

C. The Regularized Split-Gain Criterion

XGBoost only takes a split if its gain exceeds a minimum threshold γ . For a candidate split with left/right gradient and Hessian sums G_L, H_L, G_R, H_R (and parent sums G, H):

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda_{\ell_2}} + \frac{G_R^2}{H_R + \lambda_{\ell_2}} - \frac{G^2}{H + \lambda_{\ell_2}} \right] - \gamma. \quad (5)$$

A split only happens when this is positive. This is not a smooth dial: once γ exceeds the largest gain any feature can produce for a given dataset, every candidate split fails Eq. (5) and a tree stops growing at the root, collapsing the model to the training set’s base rate. We keep $\gamma \in [0.0, 1.5]$ with an ℓ_2 floor of 0.1, comfortably inside the region where trees continue to grow normally (Fig. 2).

D. Search-Space Sizing and the Optimizer’s Curse

If L_t is a hyperparameter configuration’s true generalization loss and $\hat{L}_t$ is its noisy cross-validation estimate, then for the configuration selected by $t^* = \arg \min_t \hat{L}_t$:

$$\mathbb{E}[\hat{L}_{t^*}] < \mathbb{E}[L_{t^*}], \quad (6)$$

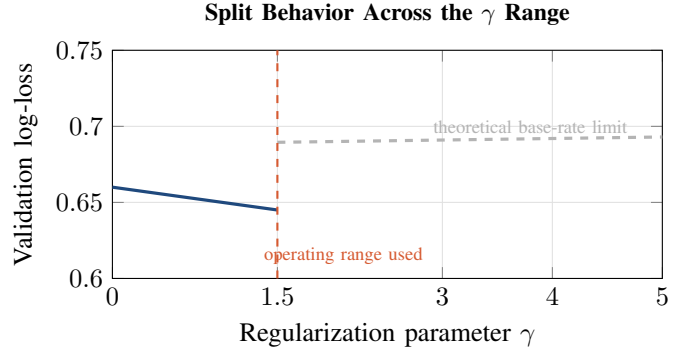


Fig. 2. Split behavior across the γ range. Loss varies smoothly within the bounded operating range; beyond the point where γ exceeds the largest feasible split gain, trees stop splitting and loss approaches the base-rate limit ($-\log 0.5 \approx 0.693$).

and the gap widens with both the number of trials T and the variance of $\hat{L}_t$ [4]. A wider search space increases the number of configurations whose cross-validation score is a favorable but unrepresentative estimate of their true generalization loss, and more trials increase the chance of finding one. We keep search bounds narrow enough ($\text{max_depth} \leq 8$, $\text{n_estimators} \leq 2000$, $\text{learning_rate} \geq 0.005$) that walk-forward folds are generalized to rather than memorized, and reserve wider bounds for periodic, explicitly labeled deep-exploration runs.

The same reasoning extends to feature count. Hughes’s classical result [6] is that for a fixed number of training rows n , expected accuracy as a function of feature count d peaks at some $d^*(n)$ and declines afterward, since the number of parameters a boosted ensemble can effectively fit grows with d while n does not. Growing the feature set in small, importance-pruned batches rather than large unpruned ones keeps the working feature count closer to $d^*(n)$ for the data actually available.

E. Stacked Generalization

On a chronologically held-out calibration slice, each base learner produces an out-of-fold probability, and a logistic regression meta-learner is fit on those probabilities as its only inputs:

$$\hat{p}_{\text{meta}} = \sigma \left(\beta_0 + \sum_{m=1}^M \beta_m \hat{p}_m \right). \quad (7)$$

Because β_m is fit on held-out data, a base learner that adds no information beyond what the others already capture, for instance one that has effectively stopped learning under a small or noisy training set, is assigned a coefficient close to zero automatically. A fixed-weight average has no equivalent mechanism: every base learner keeps a full, equal vote regardless of how little it is actually contributing. This is the precise sense in which stacking [8] weakly dominates any fixed-weight combination of the same base learners.

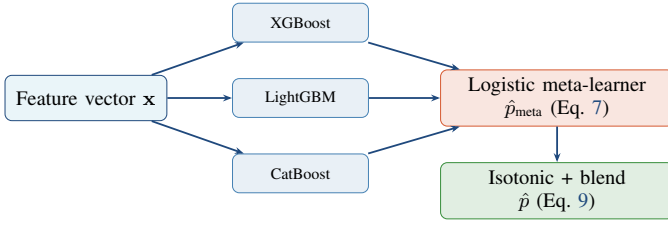


Fig. 3. The stacked-ensemble pipeline. Three gradient boosting base learners feed a logistic meta-learner, whose output is isotonic-calibrated and blended with the raw score.

F. Calibration and Variance-Preserving Blending

The meta-learner’s output is passed through isotonic regression fit on the calibration slice:

$$\hat{p}_{\text{iso}} = g(\hat{p}_{\text{meta}}), \quad g = \arg \min_{g \in \mathcal{G}_{\uparrow}} \sum_i (y_i - g(\hat{p}_{\text{meta},i}))^2, \quad (8)$$

where $\mathcal{G}_{\uparrow}$ is the set of non-decreasing step functions. Because isotonic regression is piecewise constant, a calibration set of modest size can produce wide flat segments, ranges of raw score that all map to the same output probability, which discards real per-instance variance the underlying ensemble score still carries. We therefore blend the isotonic output back with the raw score:

$$\hat{p} = \text{clip}(\alpha \hat{p}_{\text{iso}} + (1 - \alpha) \hat{p}_{\text{meta}}, \epsilon, 1 - \epsilon), \quad (9)$$

with blend weight $\alpha = 0.7$. The clip margin ϵ also matters on its own: under log-loss, a wrong prediction at probability ϵ costs $-\log \epsilon$, so $\epsilon = 10^{-6}$ costs 13.8 nats if wrong, versus roughly 0.69 nats for a prediction near an honest coin flip. We widen ϵ for smaller calibration sets specifically because that is where an isotonic fit is most likely to produce an extreme output with the least evidence behind it, keeping worst-case risk bounded without meaningfully affecting well-supported predictions.

G. Promotion as a Margin-Gated Decision

A newly trained model only replaces the current one in production if it beats the incumbent’s held-out log-loss by a fixed margin, and does not make Brier score or calibration meaningfully worse:

$$\text{promote} \iff \begin{cases} \mathcal{L}_{\text{chall}} \leq \mathcal{L}_{\text{champ}} - 0.005, \\ \text{ECE}_{\text{chall}} \leq \text{ECE}_{\text{champ}} + 0.02, \\ \text{Brier}_{\text{chall}} \leq \text{Brier}_{\text{champ}} + 0.005. \end{cases} \quad (10)$$

Requiring a fixed, meaningful margin rather than any improvement at all guards against promoting a configuration whose apparent improvement is really just the optimizer’s-curse effect of Eq. (6) at the promotion step itself.

H. Rescaling a Feature Pipeline to a Smaller Dataset

When the same feature pipeline is applied to an application with roughly half the per-season data of the largest one, across

TABLE I
SAMPLE-SIZE PARAMETER RESCALING

Parameter	Large-scale	Small-scale	Ratio
Rolling rating/pace windows	5/10/20	3/6/12	0.60
Roster-quality lookback (games)	10	5	0.50
Strength-of-schedule window	10	5	0.50
Opponent-adjustment window	15	8	0.53
Per-entity minimum sample	30	12	0.40

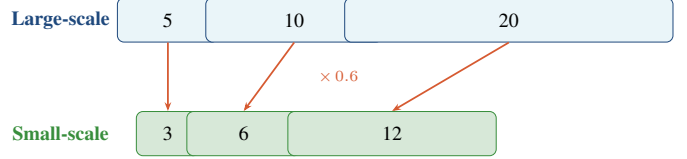


Fig. 4. Rolling-window rescaling. Bar width tracks game count within each window; small-scale windows are a uniform 60% of large-scale ones.

less than half as many distinct entities, every sample-size-dependent parameter is rescaled rather than reused unchanged. Table I shows the resulting ratios: each chosen individually based on how quickly that specific statistic stabilizes, not from one global constant.

The most aggressively rescaled parameter, the per-entity minimum sample, also gets a re-centered fallback default set to the smaller application’s own true population average rather than a value borrowed from the larger one, so an under-sampled entity regresses toward the correct population mean, standard shrinkage-estimator practice [13], [14] applied deliberately.

One new feature was added rather than just rescaled. In the smaller-scale application, output is concentrated on fewer key contributors than in the larger one, so we isolate a top-contributor availability signal, separate from the existing whole-roster availability average, and combine it with each contributor’s typical output into an expected output-at-risk term:

$$\text{AtRisk} = \text{star_output} \times (1 - \text{star_availability}). \quad (11)$$

Diluting a key contributor’s absence across a whole-roster average understates its cost; isolating it does not.

I. Selective Accuracy at a Confidence Threshold

Raw accuracy collapses genuine coin-flip predictions together with lopsided ones. The more informative measure, following the selective-classification framework [15], is accuracy conditional on confidence clearing a threshold τ :

$$\text{Acc}(\tau) = \Pr[\hat{y} = y \mid \max(\hat{p}, 1 - \hat{p}) \geq \tau]. \quad (12)$$

Section V reports this at $\tau = 0.65$ for our newest model.

V. RESULTS

Table II reports current calibration metrics for our three most mature models, computed identically across all three

TABLE II
CROSS-APPLICATION CALIBRATION METRICS

Model	Log-loss ↓	Brier ↓	ECE ↓
A (largest scale)*	0.6514	0.2179	0.0337
B*	0.6624	0.2348	0.0304
C (smallest scale)	0.6418	0.2254	0.0575

*Figures for A/B are the most recently confirmed promotions as of this writing; C reflects its current evaluation.

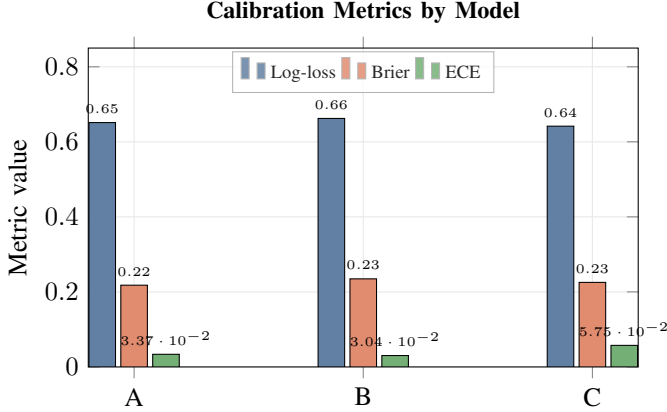


Fig. 5. Grouped comparison of Table II. Model C leads on log-loss; its higher ECE reflects a calibration curve that is still maturing relative to the longer-running models.

so the comparison is direct despite very different underlying sample sizes.

Model C, built with the rescaled small-sample pipeline of Section IV-H, reports the lowest log-loss of the three despite the smallest training set, direct evidence that the rescaling recovers signal that reusing a larger dataset’s feature parameters unchanged would have missed. Its ECE is higher than the other two, consistent with Section IV-F: a calibration curve takes longer to converge than ranking quality does, since isotonic regression needs a wide spread of predicted probabilities tested against real outcomes many times before its step function stabilizes. This is exactly why the promotion gate (Eq. (10)) checks Brier score and ECE alongside log-loss rather than optimizing log-loss alone.

At its current high-confidence operating point ($\tau = 0.65$ in Eq. (12)), Model C’s resolved predictions, meaning cases where it expressed at least 65% confidence and the outcome has since been observed, come in at 86.7% accuracy over 1,520 cases. A prediction only clears that bar after surviving the meta-learner’s combination of three independent base learners and the isotonic calibration step on top of it, so the cases that clear it are ones the whole pipeline agrees are genuinely lopsided, not just one component of it. Combined with the lowest full-sample log-loss in Table II, this is the strongest result the domain-adapted feature strategy has produced to date.

VI. DISCUSSION

The strategy and mathematics in Sections III and IV are largely domain-general. A regularized split-gain criterion, an optimizer’s-curse argument for search-space sizing, a stacking objective that lets a meta-learner discover which base learners are worth keeping, and isotonic calibration’s variance and blending tradeoffs all apply to any system built on gradient boosting, cross-validation, and automated model combination, independent of what is being predicted. The one result that is most tied to our specific setting, the domain-adapted rescaling in Section IV-H, is also the one we think generalizes the furthest as a strategy, even if the exact numbers do not: when more data is not available, a model can still be brought back to strength by encoding more of the right domain structure into its features, and the same shrinkage and effective-sample-size mathematics used throughout the rest of this paper is exactly what tells you how.

VII. CONCLUSION

We presented the strategy, the mathematics, and the measured results behind a calibrated ensemble system for outcome prediction. The strategy rests on five choices: an Elo-based strength prior, recency weighting tuned per application rather than fixed globally, a regularization search deliberately kept inside a safe operating range, a learned meta-learner combining three gradient boosting base learners in place of a fixed average, and a calibration step that blends isotonic regression’s output with the raw ensemble score to preserve per-instance variance. Each choice has a specific mathematical justification: the regularized split-gain criterion, Kish’s effective sample size, the optimizer’s-curse argument for search-space sizing, the stacking objective, and the log-loss cost of an overconfident calibrated probability. Applied to an application with roughly half the per-season data of our largest one, the same core strategy, rescaled to that application’s true sample size rather than ported from the larger one unchanged, currently produces our best-calibrated model: a log-loss of 0.6418 and 86.7% accuracy on its highest- confidence predictions across 1,520 cases. The finding we take furthest from this work is that the mathematics of effective sample size and shrinkage estimation, already load-bearing throughout the rest of the architecture, is also the right lens for deciding how to adapt a modeling strategy to a smaller dataset, and that lens should generalize well beyond the system described here.

REFERENCES

- [1] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proc. ACM SIGKDD*, pp. 785–794, 2016.
- [2] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost: Unbiased Boosting with Categorical Features,” *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [4] J. E. Smith and R. L. Winkler, “The Optimizer’s Curse: Skepticism and Postdecision Surprise in Decision Analysis,” *Management Science*, vol. 52, no. 3, pp. 311–322, 2006.

- [5] A. Gelman and E. Loken, "The Garden of Forking Paths: Why Multiple Comparisons Can Be a Problem, Even When There Is No 'Fishing Expedition' or 'p-Hacking' and the Research Hypothesis Was Posited Ahead of Time," Columbia University, Tech. Rep., 2014.
- [6] G. F. Hughes, "On the Mean Accuracy of Statistical Pattern Recognizers," *IEEE Transactions on Information Theory*, vol. 14, no. 1, pp. 55–63, 1968.
- [7] L. Kish, *Survey Sampling*. New York: Wiley, 1965.
- [8] D. H. Wolpert, "Stacked Generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [9] B. Zadrozny and C. Elkan, "Transforming Classifier Scores into Accurate Multiclass Probability Estimates," *Proc. ACM SIGKDD*, pp. 694–699, 2002.
- [10] A. Niculescu-Mizil and R. Caruana, "Predicting Good Probabilities with Supervised Learning," *Proc. ICML*, pp. 625–632, 2005.
- [11] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," *Proc. ICML*, pp. 1321–1330, 2017.
- [12] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [13] B. Efron and C. Morris, "Data Analysis Using Stein's Estimator and its Generalizations," *Journal of the American Statistical Association*, vol. 70, no. 350, pp. 311–319, 1975.
- [14] A. Gelman and J. Hill, *Data Analysis Using Regression and Multi-level/Hierarchical Models*. Cambridge University Press, 2007.
- [15] R. El-Yaniv and Y. Wiener, "On the Foundations of Noise-free Selective Classification," *Journal of Machine Learning Research*, vol. 11, pp. 1605–1641, 2010.